

Code available at:
[github.com/tidiane-camaret/
 autoML_switch_opt](https://github.com/tidiane-camaret/autoML_switch_opt)

Switching Optimizers

Arian Mousakhan¹, Hoang Dang¹, Tidiane Camaret Ndir¹, Zainab Sultan¹
 {mousakha, dangh, ndirt, soltanz}@cs.uni-freiburg.de
¹University of Freiburg

In a Nutshell

- **Observation:** Different optimizers are more suited to different topologies.
- **Idea:** design a flexible optimizer that changes its optimization technique, in accordance with whats best for the optimization problem.
- **Implementation:** using reinforcement learning, an agent is exposed to a myriad of functions - and is rewarded for the *optimal* choice of algorithm.
- **Results:** an agent that performs better than tuned standard optimizers, even on unseen tasks.

Motivation

- “**Choosing the optimizer is considered to be among the most crucial design decisions in deep learning**” (Descending through a Crowded Valley, 2021)
- Switching optimizers during the optimization process can increase the performance (Chaining of numerical black-box algorithms, 2022)
- However, evaluating the performance of every possible optimizer switch is computationally expensive
 - Idea : Train an agent to perform the switch, on a variety of problems

Design of the agent

- The agent’s design is based on 4 components:
 - **History Information:** a history of gradients and objective values encountered is provided
 - **Actions:** the agent acts in accordance with one of 3 optimizers (SGD, Adam, RMSprop)
 - **Actions Switching Mechanism:** the agent can switch between optimizers in a hard way - flip from optimizer to another or in a soft way - interpolate between optimizers via parameter manipulation
 - **Reward System:** signal is given after every step to the agent - the possible functions used are below:

$$Lookahead(action, t) = \begin{cases} 1 & \text{if } action == \text{best action after } t \text{ steps} \\ -1 & \text{Otherwise} \end{cases}$$

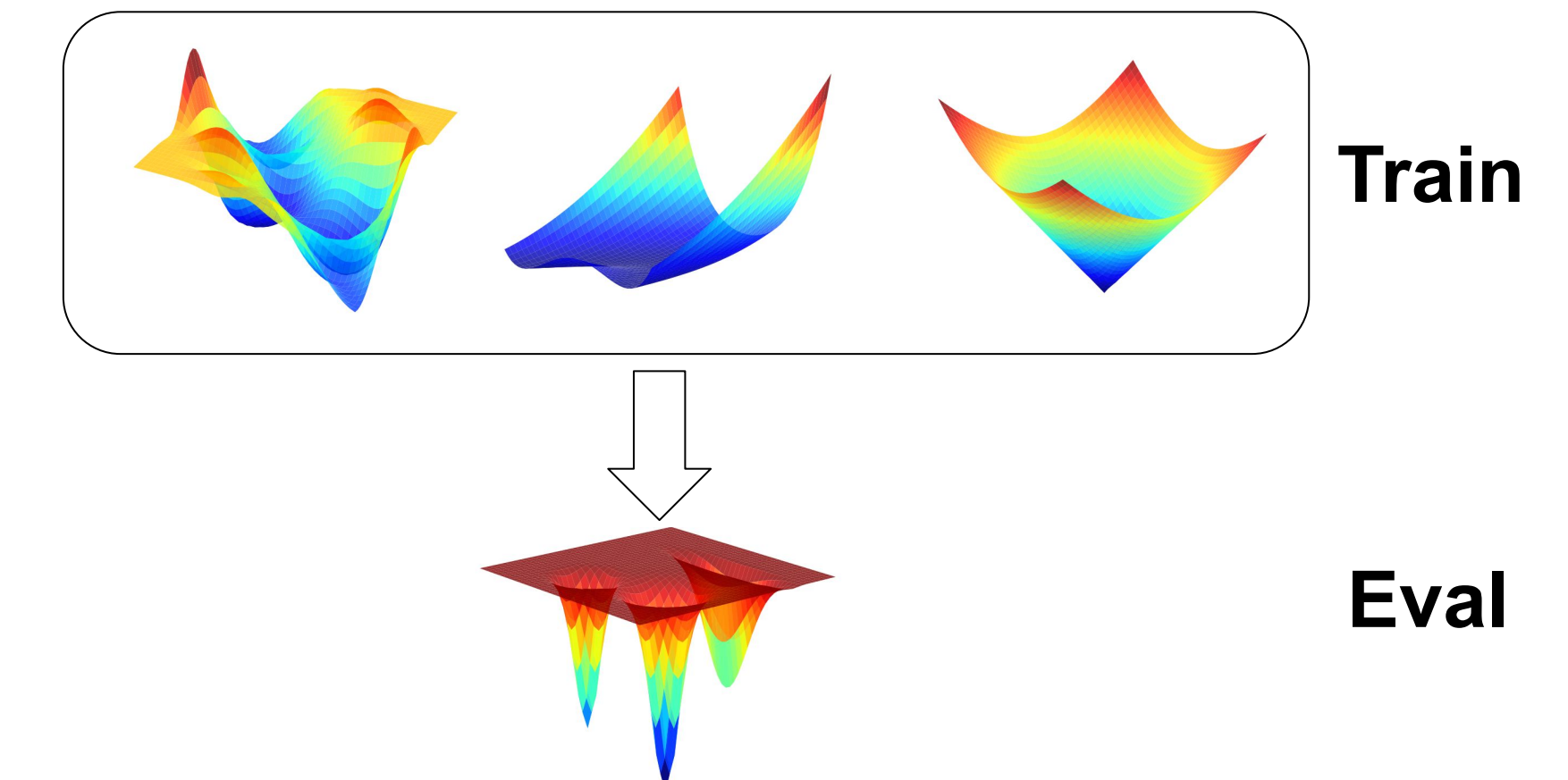
$$Threshold(objective, thres) = \begin{cases} 10 & \text{if } objective < thres \\ -1 & \text{Otherwise} \end{cases}$$

$$Opposite(objective) = -objective$$

$$Inverse(objective, thres) = \frac{1}{objective - (thres - 0.001) + 1e7}$$

Approach

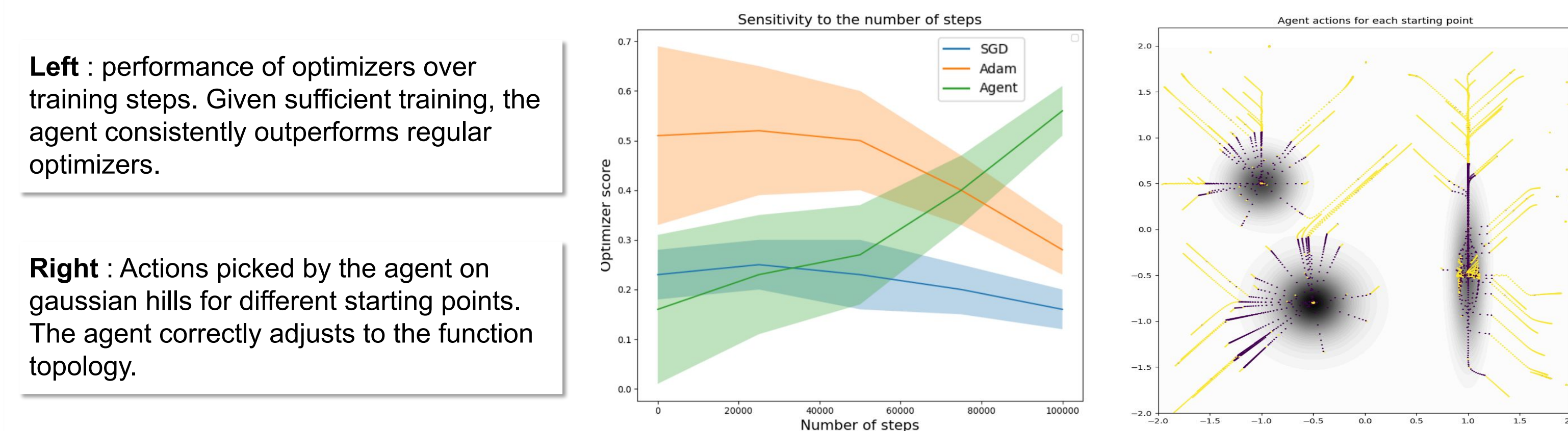
- Formulate the optimization process as a reinforcement learning environment, with which an agent can interact.
- Through interaction, the agent learns a policy that minimizes the objective
- Design optimization tasks with various topologies, so that the agent can learn a general policy



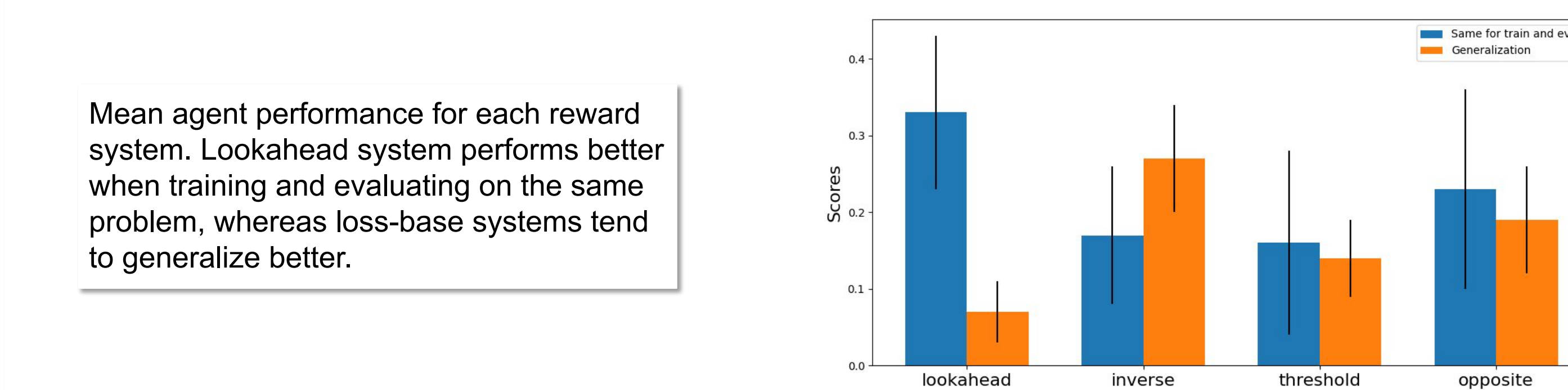
Experiments & Key Insights

White box Benchmarking: Low-dimensionality math problems

- Using the lookahead reward, the agent performs well on a single problem ...

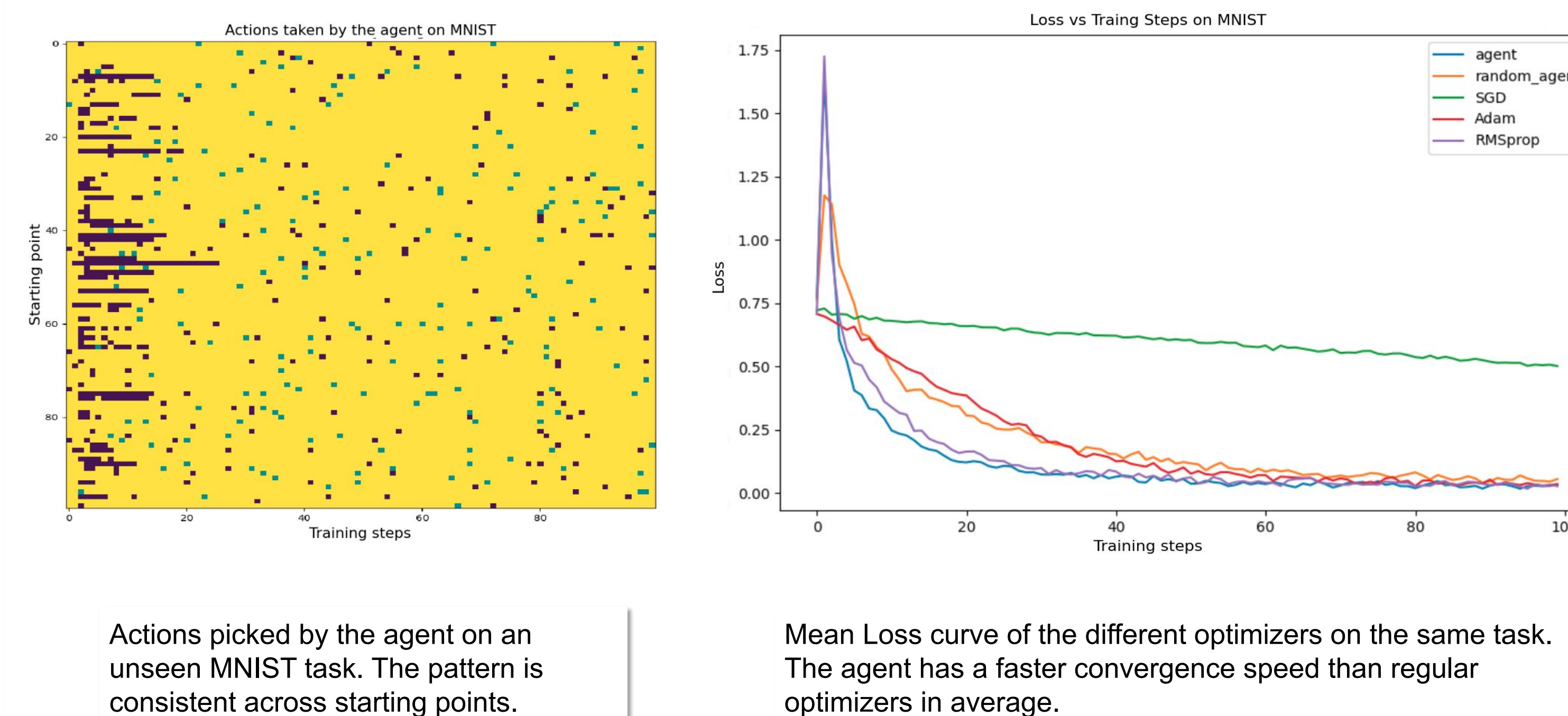


- ... But loss-based rewards generalize better to unseen problems



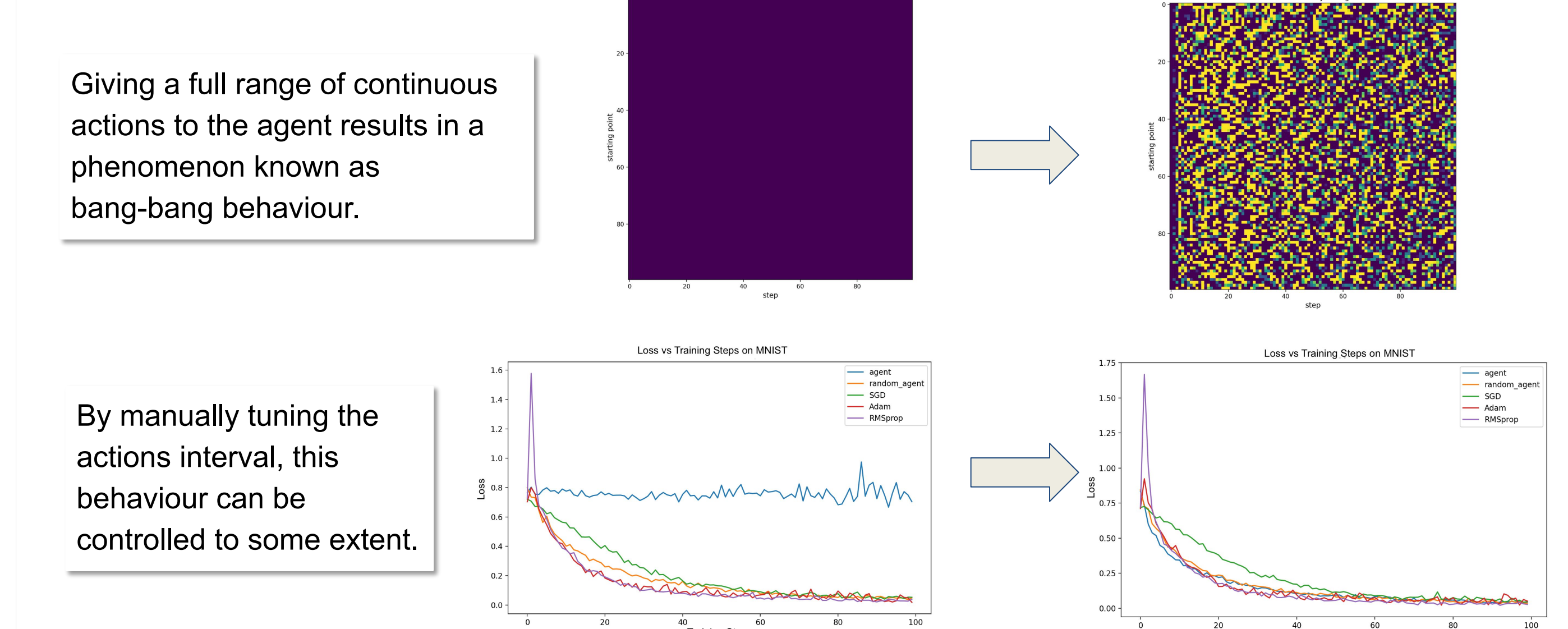
Black Box Benchmarking: Computer vision datasets

- The agent learns a switching pattern that is faster than tuned optimizers



Soft agent

- Performance of the soft agent is extremely sensitive to initial points and reward function:



Ensemble of agents

- A group of *lower budget parallelizable* agents performs better than a single agent on MNIST.

